

Correctness / Invariant

Algorithm A: FIFO Event Queue

Correctness

Algorithm A ทำงานถูกต้องตามหลัก FIFO (First In First Out) โดย Event ที่เข้ามาก่อนจะถูกนำออกจาก Queue และประมวลผลก่อน Event ที่เข้ามาทีหลัง

เมื่อใช้ enqueue() Event ใหม่จะถูกเพิ่มเข้าไปที่ด้านท้ายของ Queue และเมื่อใช้ dequeue() ระบบจะนำ Event ที่อยู่ด้านหน้าออกไปประมวลผล

Invariant

ก่อนเริ่มการประมวลผลในแต่ละรอบ สมาชิกที่อยู่บริเวณ Front ของ Queue คือ Event ที่เข้ามาก่อน Event อื่นทั้งหมดที่ยังไม่ได้รับการประมวลผล

ตัวอย่าง

Queue = [E1, E2, E3, E4]



Front

เมื่อ DEQUEUE

Output = E1

Queue = [E2, E3, E4]



Front

ดังนั้น E1 ซึ่งเข้ามาก่อนจึงถูกประมวลผลก่อน E2, E3 และ E4

สรุปความถูกต้อง

ตราบใดที่ Queue ไม่ว่าง การ dequeue() จะนำสมาชิกที่อยู่ Front ออกเสมอ ทำให้ลำดับการประมวลผลเป็นไปตามลำดับที่ Event เข้ามา จึงถูกต้องตามหลัก FIFO

Algorithm B: Priority Event Queue

Correctness

Algorithm B ทำงานถูกต้องตามหลัก **Priority Queue** โดย Event ที่มีระดับความรุนแรงสูงกว่าจะได้รับการประมวลผลก่อน

กำหนด Priority ดังนี้

Critical = 1

High = 2

Medium = 3

Low = 4

ดังนั้น Priority ที่มีค่าตัวเลขน้อยกว่าจะได้รับการประมวลผลก่อน

Invariant

ก่อนเริ่มการประมวลผลในแต่ละรอบ Event ที่ถูกเลือกจาก Priority Queue จะเป็น Event ที่มี Priority สูงที่สุดในบรรดา Event ที่ยังไม่ได้รับการประมวลผล

ตัวอย่าง

Queue

E1 = Medium = 3

E2 = Low = 4

E3 = Critical = 1

E4 = High = 2

เมื่อเลือก Event ออก ระบบจะเลือก

E3 = Critical = 1

ก่อน เพราะมี Priority สูงที่สุด

หลังจากนำ E3 ออก

E4 = High = 2

E1 = Medium = 3

E2 = Low = 4

ดังนั้น Event ถัดไปคือ E4

กรณี Priority เท่ากัน

หาก Event มี Priority เท่ากัน ให้ใช้ Sequence หรือเวลาที่รับแจ้ง เป็นตัวตัดสิน

ตัวอย่าง

E3 = Critical = 1

E5 = Critical = 1

เนื่องจาก E3 เข้ามาก่อน E5

E3 → E5

จึงถูกต้องตามเงื่อนไขของระบบ

สรุปความถูกต้อง

ตราบใดที่ Priority Queue ไม่ว่าง ระบบจะเลือก Event ที่มี Priority สูงที่สุดออกมาประมวลผลก่อนเสมอ และหากมี Priority เท่ากัน จะเลือก Event ที่เข้ามาก่อน ทำให้ลำดับการประมวลผลถูกต้องตามระดับความรุนแรงและลำดับเวลาที่รับแจ้ง

สรุป Correctness / Invariant

ประเด็น	Algorithm A: FIFO	Algorithm B: Priority
หลักการ	เข้าก่อนออกก่อน	Priority สูงออกก่อน
Invariant	Front คือ Event ที่เข้ามาก่อนที่สุด	Front คือ Event ที่มี Priority สูงที่สุด
กรณี Priority เท่ากัน	ไม่เกี่ยวข้อง	ใช้ลำดับเวลาที่เข้ามา
ความถูกต้อง	รักษาลำดับการเข้ามา	รักษาลำดับตามความรุนแรง
ผลลัพธ์ตัวอย่าง	E1 → E2 → E3 → E4 → E5 → E6	E3 → E5 → E4 → E1 → E6 → E2

Algorithm A ถูกต้องเนื่องจากรักษา Invariant ของ FIFO โดย Event ที่อยู่บริเวณ Front จะเป็น Event ที่เข้ามาก่อน Event อื่นที่ยังไม่ได้ประมวลผล จึงทำให้ Event ถูกประมวลผลตามลำดับเวลา

Algorithm B ถูกต้องเนื่องจากรักษา Invariant ของ Priority Queue โดย Event ที่มี Priority สูงที่สุดจะถูกเลือกออกมาก่อนเสมอ และหากมี Priority เท่ากัน จะพิจารณาลำดับเวลาที่รับแจ้ง เพื่อให้ Event ที่เข้ามาก่อนได้รับการประมวลผลก่อน